Playwright Notes

These notes explain the Playwright topics taught so far in training and how those topics are used in the capstone project.

The notes are arranged day-wise so that the learning path is clear:

1. What the trainer taught
2. Trainer file structure
3. Trainer code idea
4. Why the topic is useful
5. What we implemented in the capstone
6. Capstone file structure
7. Capstone code example
8. What was achieved

Capstone Website

```
https://demo.nopcommerce.com/
```

Project Goal

The capstone goal is to test the nopCommerce demo store using Playwright with JavaScript.

Capstone requirement:

- 8 services/modules
- 15 test cases per service
- 120 total test cases
- Playwright with JavaScript
- Day-wise work based on trainer topics

Basic Terms

Term	Meaning
Playwright	Tool used for browser automation testing
JavaScript	Language used to write the test scripts
Browser automation	Opening browser and performing user actions automatically
Test case	A scenario that verifies a feature
Assertion	A check that confirms expected result
Locator	Code used to find an element on a page
Spec file	Playwright Test file, usually ending with .spec.js
Config file	File that controls Playwright settings
Report	Output showing passed, failed, and skipped tests

Basic File Structure

Simple Playwright Script Structure

Used for the first Playwright day.

```
Day Folder
|-- package.json
|-- package-lock.json
|-- node_modules/
|-- test.js
`-- report.pdf
```

Playwright Test Project Structure

Used from the Playwright Test framework day onward.

```
Day Folder
|-- package.json
|-- package-lock.json
|-- node_modules/
|-- playwright.config.js
|-- tests/
|   |-- login.spec.js
|   |-- search.spec.js
|   `-- utils.js
|-- playwright-report/
|-- test-results/
`-- report.pdf
```

Important Files And Folders

File / Folder	Use
package.json	Stores project details, commands, and dependencies
package-lock.json	Locks exact installed dependency versions
node_modules/	Stores installed npm libraries
playwright.config.js	Main Playwright Test configuration
tests/	Stores test files
.spec.js	Test file extension
utils.js	Reusable helper functions
.mjs	JavaScript module file for import/export
playwright-report/	Playwright HTML report output
test-results/	Failure screenshots, traces, and error details
screenshots/	Screenshots captured in tests
.gitignore	Files/folders Git should not push

Important Dependencies And Libraries

Dependency / Library	Why It Is Used
playwright	Basic browser automation
@playwright/test	Playwright Test framework
allure-playwright	Connects Playwright with Allure report

allure-commandline	Generates and opens Allure report
express	Trainer used it for local login app practice
dotenv	Trainer used it to read environment variables
path	Node.js built-in module for file path handling

How Project Files Connect

```
package.json
    |
    | installs dependencies
    v
node_modules/

playwright.config.js
    |
    | controls test folder, browser, report, screenshots, traces
    v
tests/
    |
    | contains .spec.js files
    v
test execution
    |
    | creates report output
    v
playwright-report/
test-results/
```

In simple words:

```
package.json manages packages.
playwright.config.js manages test settings.
tests folder stores test cases.
reports folders store execution output.
```

Capstone Day 1

Trainer Reference

Trainer folder:

```
D:\WIPRO\Training\Day 12
`-- test.js
```

What Trainer Taught

The trainer taught a basic Playwright browser script.

This was the first step before using the full Playwright Test framework.

Why This Was Taught

This topic teaches how browser automation works at the simplest level.

Before writing test cases, we need to know how to:

- Open a browser
- Create a page

- Open a website
- Read the page title
- Wait
- Close the browser

Trainer Code Idea

```
const { chromium } = require('playwright');

(async () => {

  // Launch browser
  const browser = await chromium.launch({
    headless: false
  });

  // Open new page
  const page = await browser.newPage();

  // Open website
  await page.goto('https://www.google.com');

  // Print title
  console.log(await page.title());

  // Wait 5 seconds
  await page.waitForTimeout(5000);

  // Close browser
  await browser.close();

})();
```

Trainer Code Explanation

Code	Meaning
require('playwright')	Imports Playwright
chromium	Uses Chromium browser
chromium.launch()	Opens browser
headless: false	Shows browser window
browser.newPage()	Opens new tab/page
page.goto()	Opens website
page.title()	Gets page title
waitForTimeout(5000)	Waits for 5 seconds
browser.close()	Closes browser

Capstone Implementation

Capstone folder:

```
D:\WIPRO\Capstone_Project\Capstone Day 1 - Playwright Basics
|-- package.json
|-- package-lock.json
|-- node_modules/
|-- test.js
|-- run_day1.bat
`-- Day_1_Playwright_Basics_Report.pdf
```

What We Did In Capstone

We implemented the same trainer Day 12 flow on the selected capstone website.

Trainer used Google.

Capstone uses:

```
https://demo.nopcommerce.com/
```

Capstone Code File

File:

```
Capstone Day 1 - Playwright Basics/test.js
```

Code:

```
const { chromium } = require('playwright');

(async () => {

  // Launch browser
  const browser = await chromium.launch({
    headless: false
  });

  // Open new page
  const page = await browser.newPage();

  // Open website
  await page.goto('https://demo.nopcommerce.com/');

  // Print title
  console.log(await page.title());

  // Wait 5 seconds
  await page.waitForTimeout(5000);

  // Close browser
  await browser.close();

})();
```

What We Achieved

- Learned basic Playwright browser automation
- Opened the selected capstone website
- Printed the page title
- Closed the browser automatically
- Created Day 1 report PDF

Capstone Day 2

Trainer Reference

Trainer folder:

```
D:\WIPRO\Training\Day 13 & 14
|-- package.json
|-- package-lock.json
|-- playwright.config.js
|-- tests/
```

```
|   |-- example.spec.js
|   |-- utils.js
|   |-- cal.mjs
|   `-- op.mjs
`-- webapp/
    |-- index.js
    `-- public/
        |-- index.html
        |-- login.html
        `-- login.css
```

What Trainer Taught

Trainer Day 13 and Day 14 were combined.

Topics taught:

- Playwright Test framework
- test and expect
- .spec.js files
- Locators
- fill
- click
- Assertions
- Data-driven tests
- Screenshots
- Helper functions
- Import/export
- .mjs modules
- Node path module
- Local Express login app
- Form validation using fetch

Why This Was Taught

Day 1 only opened a browser.

Day 2 teaches how to write proper test cases.

This is important because a real capstone needs:

- Organized test files
- Assertions
- Reusable code
- Multiple test data
- Reports
- Project structure

Trainer Test File Idea

Trainer used:

```
import { test, expect } from '@playwright/test';
```

This imports Playwright Test framework.

Example data-driven login idea:

```
const loginData = [
  {
    username: 'tom',
    password: 'tom',
    expected: 'Username length must be greater than 3 & Password 5'
  },
  {
    username: 'admin123',
    password: 'admin123',
    expected: 'Login successful'
  }
];

loginData.forEach((data) => {
  test(`Checking Login form with ${data.username}`, async ({ page }) => {
    await page.goto('http://localhost:3000/login');

    const username = page.locator('#username');
    const password = page.locator('#password');
    const btn = page.locator('#loginBtn');
    const msg = page.locator('#msg');

    await username.fill(data.username);
    await password.fill(data.password);
    await btn.click();

    await expect(msg).toHaveText(data.expected);
  });
});
```

Trainer Helper Function Idea

Trainer used helper functions to avoid repeated code.

```
async function testLoginForm(page, user, pass) {
  const username = page.locator('#username');
  const password = page.locator('#password');
  const btn = page.locator('#loginBtn');
  const msg = page.locator('#msg');

  await username.fill(user);
  await password.fill(pass);
  await btn.click();

  return await msg.textContent();
}

export default testLoginForm;
```

Trainer Import/Export Idea

Trainer used .mjs modules.

```
function sum(a, b) {
  return a + b;
}

function sub(a, b) {
  return a - b;
}

export default sum;
```

Import example:

```
import sum from './op.mjs';
import path from 'path';
```

```
console.log(path.resolve(process.cwd()));
```

Trainer Local App Idea

Trainer used Express to create a small login app.

Main idea:

```
app.post('/loginwithcreds', (req, res) => {
  const { username, password } = req.body;

  if (username.length <= 3 || password.length <= 5) {
    res.send('Username length must be greater than 3 & Password 5');
  } else if (username === 'admin123' && password === 'admin123') {
    res.send('Login successful');
  } else {
    res.status(401).send('Invalid credentials');
  }
});
```

This taught how forms submit data and how validation messages appear.

Capstone Implementation

Capstone folder:

```
D:\WIPRO\Capstone_Project\Capstone Day 2 - Project Setup and First Tests
|-- package.json
|-- package-lock.json
|-- node_modules/
|-- playwright.config.js
|-- tests/
|   |-- login.spec.js
|   |-- search.spec.js
|   |-- module-practice.spec.js
|   |-- utils.js
|   |-- op.mjs
|   `-- cal.mjs
|-- screenshots/
|-- playwright-report/
|-- test-results/
|-- run_day2.bat
`-- Day_2_Project_Setup_and_First_Tests_Report.pdf
```

What We Did In Capstone

We applied trainer Day 13 and Day 14 learning on nopCommerce.

Implemented:

- Playwright Test setup
- Login validation test
- Search result test
- Helper functions
- Screenshot capture
- ES module import/export
- path module practice

Capstone Config File

File:

```
Capstone Day 2 - Project Setup and First Tests/playwright.config.js
```

Code idea:

```
import { defineConfig, devices } from '@playwright/test';

export default defineConfig({
  testDir: './tests',
  fullyParallel: false,
  workers: 1,
  timeout: 90000,
  expect: {
    timeout: 45000
  },
  reporter: 'html',
  use: {
    baseURL: 'https://demo.nopcommerce.com',
    headless: false,
    trace: 'on-first-retry',
    screenshot: 'only-on-failure'
  },
  projects: [
    {
      name: 'chromium',
      use: { ...devices['Desktop Chrome'] }
    }
  ]
});
```

Why We Used This Config

- baseURL avoids writing full website URL again and again.
- workers: 1 keeps tests stable on a public website.
- headless: false follows early training style where browser is visible.
- trace helps debug failed tests.
- screenshot captures evidence on failure.
- reporter: 'html' creates Playwright HTML report.

Capstone Helper File

File:

```
tests/utils.js
```

Code:

```
export async function waitForNopCommerce(page) {
  await page.waitForFunction(
    () => !/Just a moment|security verification/i.test(document.title + document.body.innerText),
    null,
    { timeout: 60000 }
  ).catch(() => {});
}

export async function testLoginForm(page, email, password) {
  const emailInput = page.locator('#Email');
  const passwordInput = page.locator('#Password');
  const loginButton = page.locator('button.login-button');

  await emailInput.fill(email);
  await passwordInput.fill(password);
  await loginButton.click();
  await waitForNopCommerce(page);

  return page.locator('body');
}
```

Why We Created Helper File

The helper file avoids writing the same login code again and again.

It also contains a wait helper because nopCommerce sometimes shows security verification.

Capstone Login Test

File:

```
tests/login.spec.js
```

Code idea:

```
import { test, expect } from '@playwright/test';
import { testLoginForm, waitForNopCommerce } from './utils.js';

const loginData = [
  {
    email: '',
    password: '',
    expected: 'Please enter your email'
  }
];

loginData.forEach((data) => {
  test(`Checking login form with email: ${data.email || 'blank email'}`, async ({ page }) => {
    await page.goto('/login');
    await waitForNopCommerce(page);

    const response = await testLoginForm(page, data.email, data.password);

    await expect(response).toContainText(data.expected);
  });
});
```

Capstone Screenshot Test

```
test('Capture login button and full page screenshots', async ({ page }) => {
  await page.goto('/login');
  await waitForNopCommerce(page);

  const loginButton = page.locator('button.login-button');

  await loginButton.screenshot({ path: 'screenshots/login-button.png' });
  await page.screenshot({ fullPage: true, path: 'screenshots/login-page-full.png' });

  await expect(loginButton).toBeVisible();
});
```

Capstone Module Practice

File:

```
tests/op.mjs
```

```
function sum(a, b) {
  return a + b;
}

function sub(a, b) {
  return a - b;
}

export default sum;
export { sub };
```

File:

```
tests/cal.mjs
```

```
import path from 'path';
import sum, { sub } from './op.mjs';
```

```
export function calculator(a, b) {
  return {
    sum: sum(a, b),
    sub: sub(a, b),
    cwd: path.resolve(process.cwd())
  };
}
```

What We Achieved

Day 2 result:

```
5 passed, 0 failed
```

We learned and implemented:

- Playwright Test project setup
- Test files
- Locators
- Fill/click/assertions
- Helper functions
- Screenshots
- Import/export
- Module practice
- Search validation

Capstone Day 3

Trainer Reference

Trainer folder:

```
D:\WIPRO\Training\Day 15
|-- package.json
|-- package-lock.json
|-- playwright.config.js
`-- tests/
    |-- example.spec.js
    `-- annotations.spec.js
```

What Trainer Taught

Trainer taught:

- test.skip
- test.fail
- test.fixme
- test.slow
- test.setTimeout
- test.step
- HTML report
- Trace config

- Browser projects
- Role locators

Why This Was Taught

After writing basic tests, we need to manage test execution and reporting.

These topics help us:

- Skip tests that are not ready
- Mark known failures
- Mark tests to fix later
- Handle slow tests
- Increase timeout
- Create readable report steps
- Generate HTML reports
- Run tests in browser projects

Trainer Annotation Code Idea

```
test.describe('Annotations and Timeouts', () => {
  test.skip('Skip this test because feature is not ready', async ({ page }) => {
    await page.goto('https://example.com');
  });

  test('Mark as expected failure', async ({ page }) => {
    test.fail();
    await page.goto('https://example.com');
    await expect(page.locator('h1')).toHaveText('Wrong Text');
  });

  test.fixme('Fix this flaky test later', async ({ page }) => {
    await page.goto('https://example.com');
  });

  test('Slow test with custom timeout', async ({ page }) => {
    test.slow();
    test.setTimeout(10000);
    await page.goto('https://playwright.dev/');
    await expect(page).toHaveTitle(/Playwright/);
  });
});
```

Trainer test.step Idea

```
test('Using test.step for reporting', async ({ page }) => {
  await test.step('Navigate to website', async () => {
    await page.goto('https://example.com');
  });

  await test.step('Verify the main heading', async () => {
    await expect(page.locator('h1')).toHaveText('Example Domain');
  });
});
```

Trainer Config Idea

```
export default defineConfig({
  testDir: './tests',
  fullyParallel: true,
  forbidOnly: !!process.env.CI,
  retries: process.env.CI ? 2 : 0,
  workers: process.env.CI ? 1 : undefined,
  reporter: 'html',
```

```
  use: {
    trace: 'on-first-retry'
  },
  projects: [
    {
      name: 'chromium',
      use: { ...devices['Desktop Chrome'] }
    },
    {
      name: 'firefox',
      use: { ...devices['Desktop Firefox'] }
    },
    {
      name: 'webkit',
      use: { ...devices['Desktop Safari'] }
    }
  ]
});
```

Capstone Implementation

Capstone folder:

```
D:\WIPRO\Capstone_Project\Capstone Day 3 - Reports Annotations and Browser Config
|-- package.json
|-- package-lock.json
|-- node_modules/
|-- playwright.config.js
|-- tests/
|   |-- annotations.spec.js
|   `-- role-locators.spec.js
|-- playwright-report/
|-- test-results/
|-- run_day3.bat
`-- Day_3_Reports_Annotations_and_Browser_Config_Report.pdf
```

What We Did In Capstone

We applied trainer Day 15 concepts on nopCommerce.

Implemented:

- Skip test
- Expected failure test
- Fixme test
- Slow test
- Custom timeout
- Step-based reporting
- Role locator test
- HTML report
- Trace and screenshot config

Capstone Annotation Test

File:

```
tests/annotations.spec.js
```

Code idea:

```
test.describe('Annotations and Timeouts on nopCommerce', () => {
  test.skip('Skip this test because future product filter scenario is not ready', async ({ page }) => {
    await page.goto('/');
```

```
  });

  test('Mark expected failure for wrong title validation', async ({ page }) => {
    test.fail();

    await page.goto('/');

    await expect(page).toHaveTitle('Wrong Title For Expected Failure');
  });

  test.fixme('Search filter scenario will be updated later', async ({ page }) => {
    await page.goto('/');
    await page.getByRole('link', { name: 'Computers' }).click();
  });
});
```

Capstone test.step Test

```
test('Using test.step for reporting on home page', async ({ page }) => {
  await test.step('Navigate to nopCommerce home page', async () => {
    await page.goto('/');
  });

  await test.step('Verify home page title', async () => {
    await expect(page).toHaveTitle(/nopCommerce demo store/);
  });

  await test.step('Verify search box is visible', async () => {
    await expect(page.locator('#small-searchterms')).toBeVisible();
  });
});
```

Capstone Role Locator Test

File:

```
tests/role-locators.spec.js
```

Code:

```
test('Use role locators on nopCommerce home page', async ({ page }) => {
  await page.goto('/');

  await expect(page.getByRole('link', { name: 'Log in' })).toBeVisible();
  await expect(page.getByRole('link', { name: 'Register' })).toBeVisible();
  await expect(page.getByRole('textbox', { name: /search/i })).toBeVisible();
  await expect(page.getByRole('button', { name: /search/i })).toBeVisible();
});
```

What We Achieved

Day 3 result:

```
4 passed, 2 skipped, 0 failed
```

We learned and implemented:

- Test annotations
- Timeouts
- Step-based reporting
- Role locators
- HTML report configuration
- Trace and screenshot configuration

Capstone Day 4

Trainer Reference

Trainer folder:

```
D:\WIPRO\Training\Day 16
|-- package.json
|-- package-lock.json
|-- playwright.config.js
`-- tests/
    `-- example.spec.js
```

What Trainer Taught

Trainer taught:

- test.describe
- beforeEach
- beforeAll
- afterEach
- afterAll
- Dialog handling
- toBeVisible
- More data-driven login tests

Why This Was Taught

When a project grows, tests need grouping and setup.

Hooks help avoid repeating the same steps.

Dialog handling is used when a website shows alert, confirm, or prompt boxes.

Trainer Code Idea

```
test.describe('testing login form', () => {
  test.beforeEach(async ({ page }) => {
    await page.goto('http://localhost:3000/login.html');
  });

  const loginData = [
    {
      username: 'tom',
      password: 'tom',
      expected: 'Username length must be greater than 3 & Password 5'
    },
    {
      username: 'admin123',
      password: 'admin123',
      expected: 'Login successful'
    }
  ];

  loginData.forEach((data) => {
    test(`Checking Login form with ${data.username}`, async ({ page }) => {
      await page.locator('#username').fill(data.username);
      await page.locator('#password').fill(data.password);
      await page.locator('#loginBtn').click();
      await expect(page.locator('#msg')).toHaveText(data.expected);
    });
  });
});
```

Dialog idea:

```
page.on('dialog', (dialog) => {
  expect(dialog.message()).toBe('Hello');
  dialog.accept();
});
```

Capstone Status

This is planned for:

```
Capstone Day 4 - Hooks Validation and Grouped Tests
```

How We Will Use It In Capstone

We will use Day 16 learning to:

- Group service-wise tests using test.describe
- Use beforeEach for repeated navigation
- Add validation tests for forms
- Use visibility assertions
- Keep tests organized by module

Capstone Day 5

Trainer Reference

Trainer folder:

```
D:\WIPRO\Training\Day 17
|-- package.json
|-- package-lock.json
|-- playwright.config.js
`-- tests/
    |-- auth.spec.js
    `-- param-assertions.spec.js
```

What Trainer Taught

Trainer taught:

- Authentication state
- storageState
- test.use
- Serial mode
- Parameterized tests
- Keyboard actions
- Regex title assertions
- Soft assertions
- Polling assertions

Why This Was Taught

Many real websites need login.

Instead of logging in again for every test, Playwright can save login state and reuse it.

Advanced assertions help verify dynamic behavior.

Trainer Authentication Code Idea

```
test.describe('auth management', () => {
  test.describe.configure({ mode: 'serial' });

  test('1. login & save state', async ({ page }) => {
    await page.goto('https://the-internet.herokuapp.com/login');
    await page.fill('#username', 'tomsmith');
    await page.fill('#password', 'SuperSecretPassword!');
    await page.click('button[type="submit"]');

    await page.context().storageState({
      path: 'playwright/.auth/user.json'
    });
  });

  test.describe('Test require login', () => {
    test.use({ storageState: 'playwright/.auth/user.json' });

    test('2. Verify secure area', async ({ page }) => {
      await page.goto('https://the-internet.herokuapp.com/secure');
      await expect(page.locator('h2')).toContainText('Secure Area');
    });
  });
});
```

Trainer Parameterized Test Idea

```
const searchData = [
  { searchTerm: 'Playwright', expectedTitle: 'Playwright' },
  { searchTerm: 'Testing', expectedTitle: 'Test' },
  { searchTerm: 'Automation', expectedTitle: 'Automation' }
];

for (const item of searchData) {
  test(`search for ${item.searchTerm}`, async ({ page }) => {
    await page.fill('input[name="search"]', item.searchTerm);
    await page.keyboard.press('Enter');
    await expect(page).toHaveTitle(new RegExp(item.expectedTitle, 'i'));
  });
}
```

Trainer Soft Assertion Idea

```
await expect.soft(page.locator('h1')).toHaveText('Example Domain');
await expect.soft(page.locator('p').first()).toBeVisible();
```

Trainer Polling Assertion Idea

```
await expect.poll(async () => {
  counter++;
  return counter;
}, {
  message: 'Counter did not reach 5',
  timeout: 5000
}).toBe(5);
```

Capstone Status

This is planned for:

```
Capstone Day 5 - Auth State and Advanced Assertions
```

How We Will Use It In Capstone

We will use Day 17 learning to:

- Save login state after customer login
- Reuse login state for account/cart/checkout flows
- Create parameterized tests for search and validations
- Use soft assertions for checking multiple product details
- Use polling assertions where values may update dynamically

Capstone Day 6

Trainer Reference

There is no separate trainer folder for this day.

Missing training day folders are considered holidays or non-training days.

Capstone Purpose

Capstone Day 6 is for review and consolidation.

Folder:

```
D:\WIPRO\Capstone_Project\Capstone Day 6 - Review Test Cases and Cleanup
```

What We Will Do

- Review completed scripts
- Review test case coverage
- Clean duplicate files
- Organize test data
- Check folder structure
- Prepare pending test cases
- Update documentation

Capstone Day 7

Trainer Reference

Trainer folder:

```
D:\WIPRO\Training\Day 19
|-- package.json
|-- package-lock.json
|-- playwright.config.js
|-- POM/
```

```
|   `-- loginPage.js
`-- tests/
    |-- login.spec.js
    `-- example.spec.js
```

What Trainer Taught

Trainer taught:

- Page Object Model
- Page classes
- Constructor locators
- Reusable page methods
- Importing POM into test file
- beforeEach object creation
- Custom fixtures
- base.extend
- preparedPage
- use(page)
- Parallel execution

Why This Was Taught

As the project grows, test files can become messy.

POM keeps locators and actions inside page classes.

Fixtures prepare common setup before tests.

Parallel execution helps run tests faster.

Trainer POM Code Idea

```
export class LoginPage {
  constructor(page) {
    this.page = page;
    this.usernameInput = page.locator('#username');
    this.passInput = page.locator('#password');
    this.btn = page.locator('#loginBtn');
  }

  async navigate() {
    await this.page.goto('http://localhost:3000/login');
  }

  async fillForm(email, pass) {
    await this.usernameInput.fill(email);
    await this.passInput.fill(pass);
  }

  async submit() {
    await this.btn.click();
  }
}
```

Trainer POM Test Idea

```
import { LoginPage } from '../POM/loginPage';

test.describe('Login tests', () => {
```

```
  let loginPage;

  test.beforeEach(async ({ page }) => {
    loginPage = new LoginPage(page);
    await loginPage.navigate();
  });

  test('Login with valid credentials', async () => {
    await loginPage.fillForm('admin', 'admin123');
    await loginPage.submit();
  });
});
```

Trainer Fixture Idea

```
const { test: base, expect } = require('@playwright/test');

const test = base.extend({
  preparedPage: async ({ page }, use) => {
    await page.goto('http://localhost:3000');
    await use(page);
  }
});

test.describe.configure({ mode: 'parallel' });
```

Capstone Status

This is planned for:

```
Capstone Day 7 - POM Fixtures and Final Run
```

How We Will Use It In Capstone

We will create page classes for:

- Home page
- Login page
- Register page
- Product page
- Cart page
- Checkout page
- Account page
- Contact page

POM will make the final project cleaner and easier to maintain.

Capstone Day 8

Trainer Reference

Trainer folder:

```
D:\WIPRO\Training\Day 21
|-- package.json
|-- package-lock.json
|-- playwright.config.js
|-- save-auth.js
|-- auth.json
|-- .gitignore
```

```
|-- tests/
|   `-- example.spec.js
|-- allure-results/
|-- playwright-report/
`-- test-results/
```

Generated folders like node_modules, allure-results, playwright-report, and test-results are output folders. The important learning files are save-auth.js, playwright.config.js, tests/example.spec.js, package.json, and .gitignore.

What Trainer Taught

Trainer Day 21 focused on:

- Allure report setup
- allure-playwright
- allure-commandline
- Playwright reporter configuration for both HTML and Allure
- allure-results
- allure-report
- Saving login session manually
- save-auth.js
- readline
- browser.newContext()
- context.storageState()
- Saving session into auth.json
- Reusing session using test.use({ storageState: 'auth.json' })
- Authenticated profile test
- .gitignore updates for generated folders and auth files

Why This Was Taught

This topic is important because many real websites require login.

If a test needs login again and again, the script becomes slow and repetitive.

Using storageState, Playwright can save cookies and local storage after login.

Then future tests can reuse the saved session.

Allure was taught because it creates a more detailed test report than the normal Playwright HTML report.

Trainer package.json Idea

Trainer added Allure dependencies:

```
{
  "devDependencies": {
    "@playwright/test": "^1.60.0",
    "@types/node": "^25.9.1",
    "allure-commandline": "^2.41.0",
    "allure-playwright": "^3.9.0"
  }
}
```

Why These Dependencies Are Used

Dependency	Use
@playwright/test	Runs Playwright Test framework
@types/node	Gives Node.js type support
allure-playwright	Connects Playwright tests with Allure
allure-commandline	Generates and opens Allure report

Trainer Allure Config Idea

File:

```
playwright.config.js
```

Important reporter line:

```
reporter: [['html'], ['allure-playwright', { resultsDir: 'allure-results' }]],
```

Meaning

- ['html'] creates Playwright HTML report.
- ['allure-playwright', { resultsDir: 'allure-results' }] creates raw Allure result files.
- allure-results is not the final report. It is the input for generating the final report.

Trainer save-auth.js Idea

File:

```
save-auth.js
```

Trainer code idea:

```
const { chromium } = require('@playwright/test');
const readline = require('readline');

function waitForEnter() {
  const rl = readline.createInterface({
    input: process.stdin,
    output: process.stdout,
  });

  return new Promise((resolve) => {
    rl.question('After you finish login in the opened browser, press Enter here to save auth.json...', () => {
      rl.close();
      resolve();
    });
  });
}

(async () => {
  const browser = await chromium.launch({ headless: false });
  const context = await browser.newContext();
  const page = await context.newPage();

  await page.goto('https://www.jiomart.com/', { waitUntil: 'domcontentloaded' });
  await waitForEnter();
  await context.storageState({ path: 'auth.json' });

  console.log('Saved login session to auth.json');
  await browser.close();
})();
```

Trainer Code Explanation

Code	Meaning
chromium.launch({ headless: false })	Opens visible browser
browser.newContext()	Creates a fresh browser context
context.newPage()	Opens page inside that context
readline	Waits for user input in terminal
waitForEnter()	Allows user to login manually before saving session
context.storageState({ path: 'auth.json' })	Saves cookies and local storage
auth.json	Stores saved login session

Why Manual Login Was Used

Some websites have OTP, captcha, QR, or security checks.

Automating login may be difficult.

So trainer opened the browser, logged in manually, then pressed Enter in the terminal.

After that, Playwright saved the logged-in session into auth.json.

Trainer Authenticated Test Idea

File:

```
tests/example.spec.js
```

Code idea:

```
import { test, expect } from '@playwright/test';

test.describe('jio mart test', () => {
  test.use({ storageState: 'auth.json' });

  test.skip('search for fruits', async ({ page }) => {
    await page.goto('https://www.jiomart.com/sections/low-price-mumbai');

    const input = page.locator('xpath-of-search-input');
    await input.click();
    await input.fill('fruits');
    await input.press('Enter');
    await expect(page.getByText('fruits').first()).toBeVisible();
  });

  test('profile', async ({ page }) => {
    await page.goto('https://www.jiomart.com/profile');

    const profileName = page.getByTestId('JDSText-text').nth(0);
    console.log(await profileName.textContent());
    await expect(profileName).toHaveText('Ajay Bhatnagar');
  });
});
```

Trainer Test Explanation

- test.use({ storageState: 'auth.json' }) tells Playwright to reuse saved login session.
- The skipped test shows a search flow planned but not executed.

- The profile test opens a protected profile page.
- The profile name is verified using toHaveText.
- getByTestId is used to locate an element by test id.

Trainer .gitignore Idea

Trainer ignored generated folders and auth session file.

```
node_modules/
/test-results/
/playwright-report/
/blob-report/
/playwright/.cache/
/playwright/.auth/
auth.json
allure-results/
allure-report/
```

Why These Are Ignored

- node_modules can be recreated using npm install.
- test-results and playwright-report are generated after tests.
- allure-results and allure-report are generated report folders.
- auth.json can contain login session data and should not be pushed to GitHub.

Capstone Status

This is the next topic to apply after the existing capstone days.

It should be added as:

```
Capstone Day 8 - Allure Reporting and Auth Session
```

How We Will Use It In Capstone

For nopCommerce, this learning can be applied to:

- Configure Allure report along with Playwright HTML report.
- Generate allure-results after test execution.
- Generate final allure-report.
- Save logged-in customer session if required.
- Reuse saved session for account, wishlist, cart, address, and checkout flows.
- Keep auth.json, allure-results, and allure-report out of GitHub.

Capstone Code Idea For Allure

In capstone playwright.config.js:

```
reporter: [['html'], ['allure-playwright', { resultsDir: 'allure-results' }]],
```

Capstone Code Idea For Auth Session

Possible capstone file:

```
save-auth.js
```

Code idea:

```
const { chromium } = require('@playwright/test');
const readline = require('readline');

function waitForEnter() {
  const rl = readline.createInterface({
    input: process.stdin,
    output: process.stdout,
  });

  return new Promise((resolve) => {
    rl.question('Login manually, then press Enter to save auth.json...', () => {
      rl.close();
      resolve();
    });
  });
}

(async () => {
  const browser = await chromium.launch({ headless: false });
  const context = await browser.newContext();
  const page = await context.newPage();

  await page.goto('https://demo.nopcommerce.com/login');
  await waitForEnter();
  await context.storageState({ path: 'auth.json' });

  await browser.close();
})();
```

What We Will Achieve

- Better reports using Allure.
- Ability to save login session.
- Ability to reuse authenticated state.
- Cleaner testing for logged-in user flows.
- Safer GitHub repository by ignoring generated and sensitive files.

Allure Reporting

What Is Allure?

Allure is an advanced reporting tool.

Playwright runs the tests.

Allure creates a detailed report from those test results.

Install Allure

```
npm install --save-dev allure-playwright allure-commandline
```

Add Allure Reporter

Inside playwright.config.js:

```
reporter: [['html'], ['allure-playwright', { resultsDir: 'allure-results' }]],
```

Run Tests

```
npx playwright test
```

Generate Allure Report

```
npx allure generate allure-results --clean -o allure-report
```

Open Allure Report

```
npx allure open allure-report
```

Allure Folders

```
allure-results/
allure-report/
```

These folders are generated output and should be added to .gitignore.

Summary Of Topics Taught So Far

- Basic Playwright browser script
- Chromium browser launch
- headless: false
- New page creation
- Website navigation
- Page title printing
- Playwright Test framework
- test and expect
- .spec.js files
- Locators
- fill
- click
- Assertions
- Data-driven tests
- Screenshots
- Helper functions
- Import/export
- .mjs files
- Node path module
- Local Express app testing
- HTML form validation
- fetch
- Playwright config

- HTML reports
- Trace and screenshot configuration
- Browser projects
- test.skip
- test.fail
- test.fixme
- test.slow
- test.setTimeout
- test.step
- Role locators
- test.describe
- Hooks
- Dialog handling
- toBeVisible
- Authentication storage state
- Serial mode
- Parameterized tests
- Keyboard actions
- Regex assertions
- Soft assertions
- Polling assertions
- Page Object Model
- Custom fixtures
- Parallel execution
- Allure reporter setup
- Manual login session saving
- auth.json
- context.storageState
- test.use({ storageState })
- getByTestId
- Allure reporting

Capstone Modules

The Playwright topics will be used to automate these capstone modules:

1. Registration and Authentication
2. Product Catalog
3. Search and Filters
4. Cart
5. Wishlist and Compare

6. Checkout and Payment
7. Address and Shipping
8. Customer Support and Account